
An allocator-aware optional type

Abstract

Library types that can potentially hold *allocator-aware* (AA) objects should, themselves, be allocator-aware. A PMR container, for example, depends on AA types following a known AA protocol so that it (the container) can uniformly manage its memory. Even types that don't manage their own memory, such as `tuple`, follow the AA rules when they hold one more more AA elements. (A special case is `pair`, which is not actually AA but effectively follows the rules through special handling in *uses-allocator construction*.)

The current definition of `std::optional` does not follow the rules for an AA type, even when holding an AA value. This limitation makes `optional` unsuitable for storage in an AA container when memory allocation customization is needed.

In this paper, we propose a new, allocator-aware `std::pmr::optional` usable as a container element type and in any other context where allocator propagation is expected. This new type would not replace the current `std::optional`, but would resolve to `std::optional` when the value type is not AA. By automatically handling the distinction between AA and non-AA value types, `pmr::optional` can easily be used in generic code where it is not known in advance if it would be storing an AA type or not.

This is a complete proposal with formal wording. It has not yet been discussed in LEWGI or LEWG.

Motivation and Proposal Summary

C++ has been moving towards a consistent PMR allocator model whereby *allocator-aware* (AA) types adhere to a few rules:

1. Once constructed, an object's allocator never changes.
2. The object's constructors all follow one of two argument protocols so that they work with *uses-allocator construction* and thus can be inserted as container elements using the container's allocator.
3. An object that can contain an AA element of user-specified type should itself be AA and use its allocator to initialize its AA subobjects.

The current `std::optional` does not follow the above rules. When disengaged, it forgets its allocator, violating the first rule; its constructors don't follow the AA protocols, so containers cannot pass their allocators to `optional` elements; and it does not hold on an allocator by which it can initialize its contained object. As a result, `std::optional` is not a good fit for situations where allocators are used. A `pmr::vector<optional<pmr::string>>`, for example, cannot ensure that all of strings within it use the same allocator, violating a key invariant of PMR containers. If one of the elements of the vector became disengaged and re-engaged, for example, the allocator for that one element could change.

The `pmr::optional` class template proposed here properly adheres to the rules of an AA type when instantiated with an AA value type. It holds a copy of the allocator used to construct it, even when disengaged, and uses that

allocator to construct its element when re-engaged. Containers of `pmr::optional` objects can correctly manage their memory.

The basic design of an AA optional is straight-forward: Add an allocator to all of its constructors and use that allocator to construct the value object each type the optional is engaged. However, when holding a non-AA type, there is no reason to pay the overhead of storing an allocator, nor to support the AA constructor interface. For this reason, `pmr::optional` is not a class template, but an alias template that resolves to an AA optional class when instantiated with an AA value type and to `std::optional` otherwise. This approach maximizes interoperability with existing `optional` while allowing programmers to use `pmr::optional` ubiquitously, even in generic code, without regard to whether or not the value type is AA. For example:

```
std::pmr::optional<int> x; // Same as std::optional
std::pmr::optional<pmr::string> y; // New, AA optional
```

Design decisions:

Rather than support the entire C++11-style set allocators, `pmr::optional` works only with `pmr::polymorphic_allocator`. The design keeps the allocator from infecting the optional's type, in the sense that it's not a template parameter. The modern PMR-style allocator model simplifies specification, implementation, and use of `pmr::optional`. Allocator-aware optionals are interoperable even if they use different concrete allocators, because the concrete allocator doesn't change the type, nor does it change the size of the type.

We want `std::pmr::optional` to work for both allocator aware and non allocator aware type. However, such type doesn't need the allocator extended interface for non allocator aware types. In fact, the interface and functionality of `std::optional` is sufficient for non allocator aware types. We propose `std::pmr::optional` being an alias template; if the element of the optional is not allocator-aware, the alias template aliases `std::optional`. Otherwise, it aliases an unspecified type that adds allocator parameters to optional's constructor overload set and which remembers the allocator when disengaged.

When prototyping this proposal, we realized that an allocator-aware optional is never trivially copyable. Copy constructor and copy assignment always potentially invoke the allocator; these operations are never bitwise copies. Move operations aren't trivial either, because they may also end up doing uses-allocator construction.

This was an implementation-based reason for having an allocator-aware type separate from the usual optional. All the complexity of conditional triviality is in `std::optional`, and `std::pmr::optional`'s allocator-aware alias target has none of that.

Feedback solicited from LEWG:

Should the general alias and the concrete allocator-aware optional type both be exposed?

This proposal doesn't expose the concrete allocator-aware optional type. `std::pmr::optional` aliases it, and we specify it, but we don't name it. Chances are that some users might want to use it as a named type, in order to reason about their code. Furthermore, it's somewhat questionable whether `std::pmr::optional<AllocAware>` and `std::pmr::optional<AllocUnaware>` should even share a name; the APIs are different, the semantics are different.

Rationale for a separate type

The reason why we chose to do it the current proposed way is that practical users just want one vocabulary type that either does or doesn't do allocators, depending on the element type's properties. The users we've talked to didn't see much value in having a type that is always allocator-aware; instead, it's allocator-aware when it can be, and otherwise not.

Which allocator should be used in value_or ?

It is not all that obvious which allocator should be used for the object returned by value_or. Should the decision be left to the type or should it be mandated by the optional ? That is, should the object be constructed by plain copy/move construction or with uses_allocator construction?

Should assignments use is_constructible trait ?

It's not construct from U we need, it's construct from U using the allocator. Do we need a new trait ?

Do we need an “allocator_aware” concept?

We might consider an “allocator aware” (AA) concept that requires a get_allocator() method that returns something convertible to polymorphic_allocator<>. If we used this concept instead of the uses_allocator trait, the allocator would require zero extra space in most (but not all) cases. (The exception would be a type that stores its allocator outside of its footprint and whose footprint is smaller than polymorphic_allocator<>.)

Proposed wording

Add new paragraphs after 20.6.1/p1 [optional.general]

[2] `pmr::optional` is an alias template such that for a type `T`, where `uses_allocator<T, polymorphic_allocator<>>` is true, `pmr::optional<T>` denotes a type which will construct the contained object with an allocator of type `polymorphic_allocator<>`. Otherwise, the alias template denotes `std::optional`.

Modify 20.6.2 Header synopsis1 [optional.syn]

```
namespace std {
    // [optional.optional], class template optional
    template<class T>
        class optional;
}
namespace pmr {
    // [alloc-optional, pmroptional], class template alloc-optional
    template<class T>
        class alloc-optional;
    template<typename T>
        using optional = conditional_t<uses_allocator<T, polymorphic_allocator<>>::value,
            alloc-optional<T>, std::optional<T>>;
}
namespace std {
    ...
```

Modify 20.6.3 Class template optional [optional.optional]

```
// [optional.ctor], constructors ...
template<class U>
    explicit(see below) optional(const pmr::alloc-optional<U>&);
template<class U>
```

explicit(see below) optional(pmr::alloc-optional<U>&&);... // [optional.assign], assignment ...

template<class U> optional& operator=(const alloc-optional&);

template<class U> optional& operator=(alloc-optional&&) noexcept(see below);

Modify 20.6.3.1 Constructors [optional.ctor]

template<class U> explicit(see below) optional(const optional<U>& v); ...

Mandates:

- is_constructible_v<T, optional<U>&> is false,
- is_constructible_v<T, alloc-optional<U>&> is false,
- is_constructible_v<T, optional<U>&&> is false,
- is_constructible_v<T, alloc-optional<U>&&> is false,
- is_constructible_v<T, const optional<U>&> is false,
- is_constructible_v<T, const alloc-optional<U>&> is false,
- is_constructible_v<T, const optional<U>&&> is false,
- is_constructible_v<T, const alloc-optional<U>&&> is false,
- is_convertible_v<optional<U>&, T> is false,
- is_convertible_v<alloc-optional<U>&, T> is false,
- is_convertible_v<optional<U>&&, T> is false,
- is_convertible_v<alloc-optional<U>&&, T> is false,
- is_convertible_v<const optional<U>&, T> is false, **and**
- is_convertible_v<const alloc-optional<U>&, T> is false,
- is_convertible_v<const optional<U>&&, T> is false, **and**
- is_convertible_v<const alloc-optional<U>&&, T> is false...

template<class U> explicit(see below) optional(optional<U>&& v); ...

Mandates:

- is_constructible_v<T, optional<U>&> is false,
- is_constructible_v<T, alloc-optional<U>&> is false,
- is_constructible_v<T, optional<U>&&> is false,
- is_constructible_v<T, alloc-optional<U>&&> is false,
- is_constructible_v<T, const optional<U>&> is false,
- is_constructible_v<T, const alloc-optional<U>&> is false,
- is_constructible_v<T, const optional<U>&&> is false,
- is_constructible_v<T, const alloc-optional<U>&&> is false,
- is_convertible_v<optional<U>&, T> is false,
- is_convertible_v<alloc-optional<U>&, T> is false,
- is_convertible_v<optional<U>&&, T> is false,
- is_convertible_v<alloc-optional<U>&&, T> is false,
- is_convertible_v<const optional<U>&, T> is false, **and**
- is_convertible_v<const alloc-optional<U>&, T> is false,
- is_convertible_v<const optional<U>&&, T> is false, **and**
- is_convertible_v<const alloc-optional<U>&&, T> is false...

...

template<class U> explicit(see below) optional(const alloc-optional<U>& rhs);

Effects: If rhs contains a value, initializes the contained value as if direct-non-list-initializing an object of type T with the expression *rhs.

Ensures: bool(rhs) == bool(*this).

Throws: Any exception thrown by the selected constructor of T.

Mandates:

- is constructible v<T, const U&> is true,
- is same<T,U> is false,
- is constructible v<T, optional<U>&> is false,
- is constructible v<T, alloc-optional<U>&> is false,
- is constructible v<T, optional<U>&&> is false,
- is constructible v<T, alloc-optional<U>&&> is false,
- is constructible v<T, const optional<U>&> is false,
- is constructible v<T, const alloc-optional<U>&> is false,
- is constructible v<T, const optional<U>&&> is false,
- is constructible v<T, const alloc-optional<U>&&> is false,
- is convertible v<optional<U>&, T> is false,
- is convertible v<alloc-optional<U>&, T> is false,
- is convertible v<optional<U>&&, T> is false,
- is convertible v<alloc-optional<U>&&, T> is false,
- is convertible v<const optional<U>&, T> is false,
- is convertible v<const alloc-optional<U>&, T> is false,
- is convertible v<const optional<U>&&, T> is false, and
- is convertible v<const alloc-optional<U>&&, T> is false

The expression inside explicit is equivalent to:

!is convertible v<const U&, T>

template<class U> explicit(see below) optional(alloc-optional<U>&& rhs);

Effects: If rhs contains a value, initializes the contained value as if direct-non-list-initializing an object of type T with the expression `std::move(*rhs)`. `std::move(*rhs)` as the constructor argument. `bool(rhs)` is unchanged.

Ensures: `bool(rhs) == bool(*this)`.

Throws: Any exception thrown by the selected constructor of T.

Remarks: The expression inside explicit is equivalent to:

!is convertible v<U, T>

Mandates:

- is constructible v<T,U&&> is true,
- is same<T,U> is false,
- is constructible v<T, optional<U>&> is false,
- is constructible v<T, alloc-optional<U>&> is false,
- is constructible v<T, optional<U>&&> is false,
- is constructible v<T, alloc-optional<U>&&> is false,
- is constructible v<T, const optional<U>&> is false,
- is constructible v<T, const alloc-optional<U>&> is false,
- is constructible v<T, const optional<U>&&> is false,
- is constructible v<T, const alloc-optional<U>&&> is false,
- is convertible v<optional<U>&, T> is false,
- is convertible v<alloc-optional<U>&, T> is false,
- is convertible v<optional<U>&&, T> is false,
- is convertible v<alloc-optional<U>&&, T> is false,
- is convertible v<const optional<U>&, T> is false,
- is convertible v<const alloc-optional<U>&, T> is false,
- is convertible v<const optional<U>&&, T> is false, and
- is convertible v<const alloc-optional<U>&&, T> is false

...

template<class U> optional& operator=(const optional<U>& rhs); ...

Remarks: ... This operator shall not participate in overload resolution unless

- is_constructible_v<T, const U> is true,
- is_assignable<T&, const U> is true,
- is_constructible_v<T, optional<U>& is false,
- is_constructible_v<T, alloc_optional<U>& is false,
- is_constructible_v<T, optional<U>&& is false,
- is_constructible_v<T, alloc_optional<U>&& is false,
- is_constructible_v<T, const optional<U>& is false,
- is_constructible_v<T, const alloc_optional<U>& is false,
- is_constructible_v<T, const optional<U>&& is false,
- is_constructible_v<T, const alloc_optional<U>&& is false,
- is_convertible_v<optional<U>&, T> is false,
- is_convertible_v<alloc_optional<U>&, T> is false,
- is_convertible_v<optional<U>&&, T> is false,
- is_convertible_v<alloc_optional<U>&&, T> is false,
- is_convertible_v<const optional<U>&, T> is false,
- is_convertible_v<const alloc_optional<U>&, T> is false,
- is_convertible_v<const optional<U>&&, T> is false,
- is_convertible_v<const alloc_optional<U>&&, T> is false,
- is_assignable_v<T&, optional<U>& is false,
- is_assignable_v<T&, alloc_optional<U>& is false,
- is_assignable_v<T&, optional<U>&& is false,
- is_assignable_v<T&, alloc_optional<U>&& is false,
- is_assignable_v<T&, const optional<U>& is false, and
- is_assignable_v<T&, const alloc_optional<U>& is false,
- is_assignable_v<T&, const optional<U>&& is false, and
- is_assignable_v<T&, const alloc_optional<U>&& is false

template<class U> optional& operator=(optional<U>&& rhs); ...

Remarks: ... This operator shall not participate in overload resolution unless

- is_constructible_v<T, const U> is true,
- is_assignable<T&, const U> is true,
- is_constructible_v<T, optional<U>& is false,
- is_constructible_v<T, alloc_optional<U>& is false,
- is_constructible_v<T, optional<U>&& is false,
- is_constructible_v<T, alloc_optional<U>&& is false,
- is_constructible_v<T, const optional<U>& is false,
- is_constructible_v<T, const alloc_optional<U>& is false,
- is_constructible_v<T, const optional<U>&& is false,
- is_constructible_v<T, const alloc_optional<U>&& is false,
- is_convertible_v<optional<U>&, T> is false,
- is_convertible_v<alloc_optional<U>&, T> is false,
- is_convertible_v<optional<U>&&, T> is false,
- is_convertible_v<alloc_optional<U>&&, T> is false,
- is_convertible_v<const optional<U>&, T> is false,
- is_convertible_v<const alloc_optional<U>&, T> is false,
- is_convertible_v<const optional<U>&&, T> is false,
- is_convertible_v<const alloc_optional<U>&&, T> is false,
- is_assignable_v<T&, optional<U>& is false,
- is_assignable_v<T&, alloc_optional<U>& is false,
- is_assignable_v<T&, optional<U>&& is false,

- is assignable `v<T&, alloc-optional<U>&&>` is false,
- is assignable `v<T&, const optional<U>&>` is false, and
- is assignable `v<T&, const alloc-optional<U>&>` is false,
- is assignable `v<T&, const optional<U>&&>` is false, and
- is assignable `v<T&, const alloc-optional<U>&&>` is false

template<class U> optional& operator=(const alloc-optional<U>& rhs);

Effects: See Table

Table — optional operator=(const alloc-optional<U>&) effects

	*this contains a value	*this does not contain a value
rhs contains a value	assigns *rhs to the contained value	initializes the contained value as if direct-non-list-initializing an object of type T with *rhs.
rhs does not contain a value	destroys the contained value by calling <code>val->T::~~T()</code> .	no effect

Returns: *this.

Ensures: `bool(rhs) == bool(*this)`.

Remarks: If any exception is thrown, the result of the expression `bool(*this)` remains unchanged. If an exception is thrown during the call to T's constructor, the state of `*rhs.val` is determined by the exception safety guarantee of T's constructor. If an exception is thrown during the call to T's assignment, the state of `*val` and `*rhs.val` is determined by the exception safety guarantee of T's assignment.

Mandates:

- is same<T,U> is false,
- is constructible `v<T, const U&>` is true,
- is assignable<T&, const U&> is true,
- is constructible `v<T, optional<U>&>` is false,
- is constructible `v<T, alloc-optional<U>&>` is false,
- is constructible `v<T, optional<U>&&>` is false,
- is constructible `v<T, alloc-optional<U>&&>` is false,
- is constructible `v<T, const optional<U>&>` is false,
- is constructible `v<T, const alloc-optional<U>&>` is false,
- is constructible `v<T, const optional<U>&&>` is false,
- is constructible `v<T, const alloc-optional<U>&&>` is false,
- is convertible `v<const U&, T>` is true,
- is convertible `v<optional<U>&, T>` is false,
- is convertible `v<alloc-optional<U>&, T>` is false,
- is convertible `v<optional<U>&&, T>` is false,
- is convertible `v<alloc-optional<U>&&, T>` is false,
- is convertible `v<const optional<U>&, T>` is false,
- is convertible `v<const alloc-optional<U>&, T>` is false,
- is convertible `v<const optional<U>&&, T>` is false,
- is convertible `v<const alloc-optional<U>&&, T>` is false,
- is assignable `v<T&, optional<U>&>` is false,
- is assignable `v<T&, alloc-optional<U>&>` is false,
- is assignable `v<T&, optional<U>&&>` is false,
- is assignable `v<T&, alloc-optional<U>&&>` is false,
- is assignable `v<T&, const optional<U>&>` is false,

- is assignable `v<T&, const alloc-optional<U>&>` is false,
- is assignable `v<T&, const optional<U>&&>` is false, and
- is assignable `v<T&, const alloc-optional<U>&&>` is false

template<class U> optional& operator=(alloc-optional<U>&& rhs);

Effects: See Table

Table — optional operator=(alloc-optional<U>&&) effects

	*this contains a value	*this does not contain a value
rhs contains a value	assigns *rhs to the contained value	initializes the contained value as if direct-non-list-initializing an object of type T with <code>std::move(*rhs)</code> .
rhs does not contain a value	destroys the contained value by calling <code>val->T::~T()</code> .	no effect

Returns: *this.

Ensures: `bool(rhs) == bool(*this)`.

Remarks: If any exception is thrown, the result of the expression `bool(*this)` remains unchanged. If an exception is thrown during the call to T's constructor, the state of `*rhs.val` is determined by the exception safety guarantee of T's constructor. If an exception is thrown during the call to T's assignment, the state of `*val` and `*rhs.val` is determined by the exception safety guarantee of T's assignment.

Mandates:

- is constructible `v<T, const U&>` is true,
- is assignable `v<T&, const U&>` is true,
- is constructible `v<T, std::optional<U>&>` is false,
- is constructible `v<T, alloc-optional<U>&>` is false,
- is constructible `v<T, std::optional<U>&&>` is false,
- is constructible `v<T, alloc-optional<U>&&>` is false,
- is constructible `v<T, const std::optional<U>&>` is false,
- is constructible `v<T, const alloc-optional<U>&>` is false,
- is constructible `v<T, const std::optional<U>&&>` is false,
- is constructible `v<T, const alloc-optional<U>&&>` is false,
- is convertible `v<const U&, T>` is true,
- is convertible `v<std::optional<U>&, T>` is false,
- is convertible `v<alloc-optional<U>&, T>` is false,
- is convertible `v<std::optional<U>&&, T>` is false,
- is convertible `v<alloc-optional<U>&&, T>` is false,
- is convertible `v<const std::optional<U>&, T>` is false,
- is convertible `v<const alloc-optional<U>&, T>` is false,
- is convertible `v<const std::optional<U>&&, T>` is false,
- is convertible `v<const alloc-optional<U>&&, T>` is false,
- is assignable `v<T&, std::optional<U>&>` is false,
- is assignable `v<T&, alloc-optional<U>&>` is false,
- is assignable `v<T&, std::optional<U>&&>` is false,
- is assignable `v<T&, alloc-optional<U>&&>` is false,
- is assignable `v<T&, const std::optional<U>&>` is false,
- is assignable `v<T&, const alloc-optional<U>&>` is false,
- is assignable `v<T&, const std::optional<U>&&>` is false, and
- is assignable `v<T&, const alloc-optional<U>&&>` is false

Header <pmroptional> synopsis [pmroptional.sys]

```
namespace pmr {
    // [alloc-optional.optional], class template alloc-optional for exposition only
    template<class T>
        class alloc-optional;

    // [optional.relops], relational operators
    template<class T, class U>
        bool operator==(const alloc-optional<T>&, const alloc-optional<U>&);
    template<class T, class U>
        bool operator==(const alloc-optional<T>&, const std::optional<U>&);
    template<class T, class U>
        bool operator==(const std::optional<T>&, const alloc-optional<U>&);
    template<class T, class U>
        bool operator!=(const alloc-optional<T>&, const alloc-optional<U>&);
    template<class T, class U>
        bool operator!=(const alloc-optional<T>&, const std::optional<U>&);
    template<class T, class U>
        bool operator!=(const std::optional<T>&, const alloc-optional<U>&);
    template<class T, class U>
        bool operator<(const alloc-optional<T>&, const alloc-optional<U>&);
    template<class T, class U>
        bool operator<(const alloc-optional<T>&, const std::optional<U>&);
    template<class T, class U>
        bool operator<(const std::optional<T>&, const alloc-optional<U>&);
    template<class T, class U>
        bool operator>(const alloc-optional<T>&, const alloc-optional<U>&);
    template<class T, class U>
        bool operator>(const alloc-optional<T>&, const std::optional<U>&);
    template<class T, class U>
        bool operator>(const std::optional<T>&, const alloc-optional<U>&);
    template<class T, class U>
        bool operator<=(const alloc-optional<T>&, const alloc-optional<U>&);
    template<class T, class U>
        bool operator<=(const alloc-optional<T>&, const std::optional<U>&);
    template<class T, class U>
        bool operator<=(const std::optional<T>&, const alloc-optional<U>&);
    template<class T, class U>
        bool operator>=(const alloc-optional<T>&, const alloc-optional<U>&);
    template<class T, class U>
        bool operator>=(const alloc-optional<T>&, const std::optional<U>&);
    template<class T, class U>
        bool operator>=(const std::optional<T>&, const alloc-optional<U>&);
    template<class T, three way comparable with<T> U>
        compare_three_way_result t<T,U>
            operator<=>(const alloc-optional<T>&, const alloc-optional<U>&);
    template<class T, three way comparable with<T> U>
        compare_three_way_result t<T,U>
            operator<=>(const alloc-optional<T>&, const std::optional<U>&);
    template<class T, three way comparable with<T> U>
        compare_three_way_result t<T,U>
            operator<=>(const std::optional<T>&, const alloc-optional<U>&);

    // [alloc-optional.nullops], comparison with nullopt
    template<class T> bool operator==(const alloc-optional<T>&, nullopt t)
noexcept;
    template<class T>
        strong_ordering operator<=>(const alloc-optional<T>&, nullopt t) noexcept;

    // [alloc-optional.comp.with.t], comparison with T
    template<class T, class U> bool operator==(const alloc-optional<T>&, const U&);
    template<class T, class U> bool operator==(const T&, const alloc-optional<U>&);
    template<class T, class U> bool operator!=(const alloc-optional<T>&, const U&);
```

```

template<class T, class U> bool operator!=(const T&, const alloc-optional<U>&);
template<class T, class U> bool operator<(const alloc-optional<T>&, const U&);
template<class T, class U> bool operator<(const T&, const alloc-optional<U>&);
template<class T, class U> bool operator>(const alloc-optional<T>&, const U&);
template<class T, class U> bool operator>(const T&, const alloc-optional<U>&);
template<class T, class U> bool operator<=(const alloc-optional<T>&, const U&);
template<class T, class U> bool operator<=(const T&, const alloc-optional<U>&);
template<class T, class U> bool operator>=(const alloc-optional<T>&, const U&);
template<class T, class U> bool operator>=(const T&, const alloc-optional<U>&);
template<class T, three way comparable with<T> U>
    compare_three_way_result t<T,U>
        operator<=>(const alloc-optional<T>&, const U&);

// [alloc-optional.specalg], specialized algorithms
template<class T>
    void swap(alloc-optional<T>&, alloc-optional<T>&) noexcept(see below);

template<class T>
    constexpr optional<see below> make_optional(T&&);
template<class T, class... Args>
    constexpr optional<T> make_optional(Args&&... args);
template<class T, class U, class... Args>
    constexpr optional<T> make_optional(initializer_list<U> il, Args&&... args);

template<class T>
    constexpr optional<see below> alloc_optional(const polymorphic_allocator<>&
a, T&&);
template<class T, class... Args>
    constexpr optional<T> alloc_optional(const polymorphic_allocator<>& a,
Args&&... args);
template<class T, class U, class... Args>
    constexpr optional<T> alloc_optional(const polymorphic_allocator<>& a,
initializer_list<U> il, Args&&... args);

// [alloc-optional.hash], hash support
template<class T> struct hash<alloc-optional<T>>;
}

```

Class template *alloc-optional* [alloc-optional.pmroptional]

```

namespace std::pmr {
    template<class T>
        class alloc-optional {
        public:
            using value_type = T;
            using allocator_type = std::pmr::polymorphic_allocator<>;

            // [alloc-optional.ctor], constructors
            alloc-optional() noexcept;
            alloc-optional(nullopt_t) noexcept;
            alloc-optional(const alloc-optional&);
            alloc-optional(alloc-optional&&) noexcept(see below);
            template<class... Args>
                explicit alloc-optional(in_place_t, Args&&...);
            template<class U, class... Args>
                explicit alloc-optional(in_place_t, initializer_list<U>, Args&&...);
            template<class U = T>
                explicit(see below) alloc-optional(U&&);
            template<class U>
                explicit(see below) alloc-optional(const alloc-optional<U>&);
            template<class U>
                explicit(see below) alloc-optional(alloc-optional<U>&&);
            template<class U>
                explicit(see below) alloc-optional(const std::optional<U>&);

```

```

    template<class U>
        explicit(see below) alloc-optional(std::optional<U>&&);

    // allocator-extended constructors
    alloc-optional(allocator_arg_t, const polymorphic_allocator<>&) noexcept;
    alloc-optional(allocator_arg_t, const polymorphic_allocator<>&, nullopt_t)
noexcept;
    alloc-optional(allocator_arg_t, const polymorphic_allocator<>&, const alloc-
optional&);
    alloc-optional(allocator_arg_t, const polymorphic_allocator<>&, alloc-
optional&&) noexcept(see below);
    template<class... Args>
        explicit alloc-optional(allocator_arg_t, const polymorphic_allocator<>&,
in_place_t, Args&&...);
    template<class U, class... Args>
        explicit alloc-optional(allocator_arg_t, const polymorphic_allocator<>&,
in_place_t, initializer_list<U>, Args&&...);
    template<class U = T>
        explicit(see below) alloc-optional(allocator_arg_t, const
polymorphic_allocator<>&, U&&);
    template<class U>
        explicit(see below) alloc-optional(allocator_arg_t, const
polymorphic_allocator<>&, const alloc-optional<U>&);
    template<class U>
        explicit(see below) alloc-optional(allocator_arg_t, const
polymorphic_allocator<>&, alloc-optional<U>&&);
    template<class U>
        explicit(see below) alloc-optional(allocator_arg_t, const
polymorphic_allocator<>&, const std::optional<U>&);
    template<class U>
        explicit(see below) alloc-optional(allocator_arg_t, const
polymorphic_allocator<>&, std::optional<U>&&);

    // [alloc-optional.dtor], destructor
    ~alloc-optional();

    // [alloc-optional.assign], assignment
    alloc-optional& operator=(nullopt_t) noexcept;
    alloc-optional& operator=(const alloc-optional&);
    alloc-optional& operator=(alloc-optional&&) noexcept(see below);
    template<class U = T> alloc-optional& operator=(U&&);
    template<class U> alloc-optional& operator=(const alloc-optional<U>&&);
    template<class U> alloc-optional& operator=(alloc-optional<U>&&);
    template<class U> alloc-optional& operator=(const std::optional<U>&&);
    template<class U> alloc-optional& operator=(std::optional<U>&&);
    template<class... Args>> T& emplace(Args&&...);
    template<class U, class... Args> T& emplace(initializer_list<U>, Args&&...);

    // [alloc-optional.swap], swap
    void swap(alloc-optional&) noexcept(see below);

    // [alloc-optional.observe], observers
    const T* operator->() const;
    T* operator->();
    const T& operator*() const&;
    T& operator*() &;
    T&& operator*() &&;
    const T&& operator*() const&&;
    explicit operator bool() const noexcept;
    bool has_value() const noexcept;
    const T& value() const&;
    T& value() &;
    T&& value() &&;
    const T&& value() const&&;
    template<class U> T value_or(U&&) const&;

```

```

        template<class U> T value_or(U&&) &&;
        template<class U> T value_or(allocator_arg_t, const
polymorphic_allocator<>&, U&&) const&;
        template<class U> T value_or(allocator_arg_t, const
polymorphic_allocator<>&, U&&) &&;
        allocator_type get_allocator() const noexcept;

// [alloc-optional.mod], modifiers
void reset() noexcept;

private:
    T* val; // exposition only
    const std::memory_resource* M_pmr; // exposition only
};

/* todo CTAD */
}

```

Any instance of *alloc-optional*<T> at any given time either contains a value or does not contain a value. When an instance of *alloc-optional*<T> contains a value, it means that an object of type T, referred to as the *alloc-optional* object's contained value, is allocated within the storage of the *alloc-optional* object. Implementations are not permitted to use additional storage, such as dynamic memory, to allocate its contained value. The contained value shall be allocated in a region of the *alloc-optional*<T> storage suitably aligned for the type T and constructed using M_pmr. When an object of type *alloc-optional*<T> is contextually converted to bool, the conversion returns true if the object contains a value; otherwise the conversion returns false.

Member val is provided for exposition only. When an *alloc-optional*<T> object contains a value, val points to the contained value.

Member M_pmr is provided for exposition only and points to the memory_resource used when creating the contained value. If the value type provides get_allocator() member function, and its return type is convertible to polymorphic_allocator<>, the implementation is allowed to assume that the allocator used for constructing the element is the same allocator as returned by T::get_allocator(). [Note: In such an implementation, pmr::optional<pmr::string> will be zero-overhead compared to optional<pmr::string>. -- end note]

T shall be a type other than cv in_place_t, cv allocator_arg_t, or cv nullopt_t that meets the Cpp17Destructible requirements (Table 30).

Constructors [alloc-optional.ctor]

```

alloc-optional() noexcept;
alloc-optional(nullopt_t) noexcept;
    Effects: M_pmr is initialized with std::pmr::get_default_resource().
    Ensures: *this does not contain a value.
    Remarks: No contained value is initialized.

```

```

alloc-optional(const alloc-optional&);
    Effects: M_pmr is initialized with std::pmr::get_default_resource(). If rhs contains a
value, initializes the contained value with the expression *rhs.
    Ensures: bool(rhs) == bool(*this).
    Throws: Any exception thrown by the selected constructor of T.
    Remarks: This constructor shall be defined as deleted unless is_copy_constructible v<T> is
true.

```

`alloc-optional(alloc-optional&& rhs) noexcept(see below)`

Effects: `M_pmr` is initialized with `rhs.M_pmr`. If `rhs` contains a value, initializes the contained value with the expression `std::move(*rhs)`. `bool(rhs)` is unchanged.

Ensures: `bool(rhs) == bool(*this)`.

Throws: Any exception thrown by the selected constructor of `T`.

Remarks: The expression inside `noexcept` is equivalent to `is_nothrow_move_constructible_v<T>`.

Mandates: `is_move_constructible_v<T>` is true.

`template<class... Args> explicit alloc-optional(in_place t, Args&&... args);`

Effects: `M_pmr` is initialized with `std::pmr::get_default_resource()`. If `rhs` contains a value, initializes the contained value by uses-allocator construction with allocator `polymorphic_allocator<>(_M_pmr)` and `std::forward<Args>(args)...` as the constructor arguments.

Ensures: `*this` contains a value..

Throws: Any exception thrown by the selected constructor of `T`.

Mandates: `is_copy_constructible_v<T>` is true.

`template<class... Args> explicit alloc-optional(in_place t, initializer_list<U> il, Args&&... args);`

Effects: `M_pmr` is initialized with `std::pmr::get_default_resource()`. If `rhs` contains a value, initializes the contained value by uses-allocator construction with allocator `polymorphic_allocator<>(_M_pmr)` and `il, std::forward<Args>(args)...` as the constructor arguments.

Ensures: `*this` contains a value..

Throws: Any exception thrown by the selected constructor of `T`.

Mandates: `is_constructible_v<T, initializer_list<U>&, Args&&...>` is true.

`template<class U = T> explicit(see below) alloc-optional(U&& rhs);`

Effects: `M_pmr` is initialized with `std::pmr::get_default_resource()`. If `rhs` contains a value, initializes the contained value by uses-allocator construction with allocator `polymorphic_allocator<>(_M_pmr)` and `std::forward<U>(rhs)` as the constructor arguments.

Ensures: `*this` contains a value.

Throws: Any exception thrown by the selected constructor of `T`.

Remarks: The expression inside `explicit` is equivalent to:

`!is_convertible_v<U, T>`

Mandates:

- `is_constructible_v<T, U&&>` is true,

- `is_same_v<remove_cvref_t<U>, in_place_t>` is false, and

- `is_same_v<remove_cvref_t<U>, alloc-optional>` is false.

`template<class U> explicit(see below) alloc-optional(const alloc-optional<U>& rhs);`

Effects: `M_pmr` is initialized with `std::pmr::get_default_resource()`. If `rhs` contains a value, initializes the contained value by uses-allocator construction with allocator `polymorphic_allocator<>(_M_pmr)` and `*rhs` as the constructor argument.

Ensures: `bool(rhs) == bool(*this)`.

Throws: Any exception thrown by the selected constructor of `T`.

Remarks: The expression inside `explicit` is equivalent to:

`!is_convertible_v<const U&, T>`

Mandates:

- `is_constructible_v<T, const U&>` is true,

- `is_same<T, U>` is false,

- `is_constructible_v<T, std::optional<U>&>` is false,

- `is_constructible_v<T, alloc-optional<U>&>` is false,

- is constructible `v<T, std::optional<U>&&>` is false,
- is constructible `v<T, alloc_optional<U>&&>` is false,
- is constructible `v<T, const std::optional<U>&>` is false,
- is constructible `v<T, const alloc_optional<U>&>` is false,
- is constructible `v<T, const std::optional<U>&&>` is false,
- is constructible `v<T, const alloc_optional<U>&&>` is false,
- is convertible `v<std::optional<U>&, T>` is false,
- is convertible `v<alloc_optional<U>&, T>` is false,
- is convertible `v<std::optional<U>&&, T>` is false,
- is convertible `v<alloc_optional<U>&&, T>` is false,
- is convertible `v<const std::optional<U>&, T>` is false,
- is convertible `v<const alloc_optional<U>&, T>` is false,
- is convertible `v<const std::optional<U>&&, T>` is false, and
- is convertible `v<const alloc_optional<U>&&, T>` is false

`template<class U> explicit(see below) alloc_optional(alloc_optional<U>&& rhs);`

Effects: `M_pmr` is initialized with `std::pmr::get_default_resource()`. If `rhs` contains a value, initializes the contained value by uses-allocator construction with allocator `polymorphic_allocator<>(_M_pmr)` and `std::move(*rhs)` as the constructor argument. `bool(rhs)` is unchanged.

Ensures: `bool(rhs) == bool(*this)`.

Throws: Any exception thrown by the selected constructor of `T`.

Remarks: The expression inside `explicit` is equivalent to:

Mandates:

- `!is_convertible<v<U, T>`
- is constructible `v<T, U&&>` is true,
- is `same<T, U>` is false,
- is constructible `v<T, std::optional<U>&>` is false,
- is constructible `v<T, alloc_optional<U>&>` is false,
- is constructible `v<T, std::optional<U>&&>` is false,
- is constructible `v<T, alloc_optional<U>&&>` is false,
- is constructible `v<T, const std::optional<U>&>` is false,
- is constructible `v<T, const alloc_optional<U>&>` is false,
- is constructible `v<T, const std::optional<U>&&>` is false,
- is constructible `v<T, const alloc_optional<U>&&>` is false,
- is convertible `v<std::optional<U>&, T>` is false,
- is convertible `v<alloc_optional<U>&, T>` is false,
- is convertible `v<std::optional<U>&&, T>` is false,
- is convertible `v<alloc_optional<U>&&, T>` is false,
- is convertible `v<const std::optional<U>&, T>` is false,
- is convertible `v<const alloc_optional<U>&, T>` is false,
- is convertible `v<const std::optional<U>&&, T>` is false, and
- is convertible `v<const alloc_optional<U>&&, T>` is false

`template<class U> explicit(see below) alloc_optional(const std::optional<U>& rhs);`

Effects: `M_pmr` is initialized with `std::pmr::get_default_resource()`. If `rhs` contains a value, initializes the contained value by uses-allocator construction with allocator `polymorphic_allocator<>(_M_pmr)` and `*rhs` as the constructor argument.

Ensures: `bool(rhs) == bool(*this)`.

Throws: Any exception thrown by the selected constructor of `T`.

Remarks: The expression inside `explicit` is equivalent to:

`!is_convertible<v<const U&, T>`

Mandates:

- is constructible `v<T, const U&>` is true,
- is constructible `v<T, std::optional<U>&>` is false,
- is constructible `v<T, alloc-optional<U>&>` is false,
- is constructible `v<T, std::optional<U>&&>` is false,
- is constructible `v<T, alloc-optional<U>&&>` is false,
- is constructible `v<T, const std::optional<U>&>` is false,
- is constructible `v<T, const alloc-optional<U>&>` is false,
- is constructible `v<T, const std::optional<U>&&>` is false,
- is constructible `v<T, const alloc-optional<U>&&>` is false,
- is convertible `v<std::optional<U>&, T>` is false,
- is convertible `v<alloc-optional<U>&, T>` is false,
- is convertible `v<std::optional<U>&&, T>` is false,
- is convertible `v<alloc-optional<U>&&, T>` is false,
- is convertible `v<const std::optional<U>&, T>` is false,
- is convertible `v<const alloc-optional<U>&, T>` is false,
- is convertible `v<const std::optional<U>&&, T>` is false, and
- is convertible `v<const alloc-optional<U>&&, T>` is false

`template<class U> explicit(see below) alloc-optional(std::optional<U>&& rhs);`
Effects: `M_pmr` is initialized with `std::pmr::get_default_resource()`. If `rhs` contains a value, initializes the contained value by uses-allocator construction with allocator `polymorphic_allocator<>(M_pmr)` and `std::move(*rhs)` as the constructor argument. `bool(rhs)` is unchanged.

Ensures: `bool(rhs) == bool(*this)`.

Throws: Any exception thrown by the selected constructor of `T`.

Remarks:

The expression inside `explicit` is equivalent to:

`!is_convertible_v<U, T>`

Mandates:

- is constructible `v<T, U&&>` is true,
- is constructible `v<T, std::optional<U>&>` is false,
- is constructible `v<T, alloc-optional<U>&>` is false,
- is constructible `v<T, std::optional<U>&&>` is false,
- is constructible `v<T, alloc-optional<U>&&>` is false,
- is constructible `v<T, const std::optional<U>&>` is false,
- is constructible `v<T, const alloc-optional<U>&>` is false,
- is constructible `v<T, const std::optional<U>&&>` is false,
- is constructible `v<T, const alloc-optional<U>&&>` is false,
- is convertible `v<std::optional<U>&, T>` is false,
- is convertible `v<alloc-optional<U>&, T>` is false,
- is convertible `v<std::optional<U>&&, T>` is false,
- is convertible `v<alloc-optional<U>&&, T>` is false,
- is convertible `v<const std::optional<U>&, T>` is false,
- is convertible `v<const alloc-optional<U>&, T>` is false,
- is convertible `v<const std::optional<U>&&, T>` is false, and
- is convertible `v<const alloc-optional<U>&&, T>` is false

`alloc-optional(allocator arg t, const polymorphic_allocator<>&) noexcept;`
`alloc-optional(allocator arg t, const polymorphic_allocator<>&, nullopt t) noexcept;`
`alloc-optional(allocator arg t, const polymorphic_allocator<>&, const alloc-optional&);`
`alloc-optional(allocator arg t, const polymorphic_allocator<>&, alloc-optional&&) noexcept(see below);`


```

template<class... Args>
    explicit alloc_optional(allocator arg t, const polymorphic allocator<>&,
in place t, Args&&...);
template<class U, class... Args>
    explicit alloc_optional(allocator arg t, const polymorphic allocator<>&,
in place t, initializer_list<U>, Args&&...);
template<class U = T>
    explicit(see below) alloc_optional(allocator arg t, const
polymorphic allocator<>&,U&&);
template<class U>
    explicit(see below) alloc_optional(allocator arg t, const
polymorphic allocator<>&,const alloc_optional<U>&);
template<class U>
    explicit(see below) alloc_optional(allocator arg t, const
polymorphic allocator<>&,alloc_optional<U>&&);
template<class U>
    explicit(see below) alloc_optional(allocator arg t, const
polymorphic allocator<>&,const std::optional<U>&);
template<class U>
    explicit(see below) alloc_optional(allocator arg t, const
polymorphic allocator<>&,std::optional<U>&&);
    Constraints: uses allocator<T,polymorphic allocator<>> is true;
    Effects: Equivalent to the preceding constructors except that M_pmr is initialized with
a.resource().

```

Destructor [alloc_optional.dtor]

```
~alloc_optional();
```

Effects: If is trivially destructible $v<T> != \text{true}$ and **this* contains a value, calls $\text{val->T::}\sim T()$.

Remarks: If is trivially destructible $v<T>$ is true, then this destructor is trivial.

Assignment [alloc_optional.assign]

```
alloc_optional<T>& operator=(nullopt t) noexcept;
```

Effects: If **this* contains a value, calls $\text{val->T::}\sim T()$ to destroy the contained value; otherwise no effect.

Remarks: **this*

Ensures: **this* does not contain a value.

```
alloc_optional<T>& operator=(const alloc_optional& rhs);
```

Effects: See Table

Table — *alloc_optional* operator=(const *alloc_optional*& rhs) effects

	<i>*this</i> contains a value	<i>*this</i> does not contain a value
rhs contains a value	assigns <i>*rhs</i> to the contained value	initializes the contained value by uses-allocator construction with allocator polymorphic allocator<>(_M_pmr) and <i>*rhs</i> as the constructor argument.
rhs does not contain a value	destroys the contained value by calling $\text{val->T::}\sim T()$.	no effect

Returns: *this.

Ensures: bool(rhs) == bool(*this).

Remarks: If any exception is thrown, the result of the expression bool(*this) remains unchanged. If an exception is thrown during the call to T's copy constructor, no effect. If an exception is thrown during the call to T's copy assignment, the state of of its contained value is as defined by the exception safety guarantee of is determined by the exception safety guarantee of T's copy assignment. This operator shall be defined as deleted unless is copy constructible v<T> is true and is copy assignable v<T> is true.

alloc-optional<T>& operator=(alloc-optional&& rhs) noexcept(see below);

Effects: See Table

Table — alloc-optional operator=(alloc-optional&& rhs) effects

	<u>*this contains a value</u>	<u>*this does not contain a value</u>
<u>rhs contains a value</u>	<u>assigns std::move(*rhs) to the contained value</u>	<u>initializes the contained value by uses-allocator construction with allocator polymorphic allocator<>(_M_pmr) and std::move(*rhs) as the constructor argument.</u>
<u>rhs does not contain a value</u>	<u>destroys the contained value by calling val->T::~T(.).</u>	<u>no effect</u>

Returns: *this.

Ensures: bool(rhs) == bool(*this).

Remarks: The expression inside noexcept is equivalent to:

is nothrow move assignable v<T> && is nothrow move constructible v<T>

If any exception is thrown, the result of the expression bool(*this) remains unchanged. If an exception is thrown during the call to T's move constructor, the state of *rhs.val is determined by the exception safety guarantee of T's move constructor. If an exception is thrown during the call to T's move assignment, the state of *val and *rhs.val is determined by the exception safety guarantee of T's move assignment.

Mandates: is move constructible v<T> is true and is move assignable v<T> is true.

template<class U = T> alloc-optional& operator=(U&& v);

Effects: If *this contains a value, assigns std::forward<U>(v) to the contained value; otherwise initializes the contained value by uses-allocator construction with allocator polymorphic allocator<>(_M_pmr) and std::forward<U>(v) as the constructor argument.

Returns: *this.

Ensures: *this contains a value.

Remarks: If any exception is thrown, the result of the expression bool(*this) remains unchanged. If an exception is thrown during the call to T's constructor, the state of v is determined by the exception safety guarantee of T's constructor. If an exception is thrown during the call to T's assignment, the state of *val and v is determined by the exception safety guarantee of T's assignment.

Mandates:

- is same v<remove cvref t<U>, alloc-optional> is false,

- conjunction v<is scalar<T>, is same<T, decay t<U>>> is false,

- is constructible v<T,U> is true, and

- is assignable v<T&,U> is true

```
template<class U> alloc-optional& operator=(const alloc-optional<U>& rhs);
```

Effects: See Table

Table — alloc-optional operator=(const alloc-optional<U>&) effects

	<u>*this contains a value</u>	<u>*this does not contain a value</u>
<u>rhs contains a value</u>	<u>assigns *rhs to the contained value</u>	<u>initializes the contained value by uses-allocator construction with allocator polymorphic allocator<>(_M_pmr) and *rhs as the constructor argument.</u>
<u>rhs does not contain a value</u>	<u>destroys the contained value by calling val->T::~~T().</u>	<u>no effect</u>

Returns: *this.

Ensures: bool(rhs) == bool(*this).

Remarks: If any exception is thrown, the result of the expression bool(*this) remains unchanged. If an exception is thrown during the call to T's constructor, no effect. If an exception is thrown during the call to T's assignment, the state of *val is determined by the exception safety guarantee of T's assignment.

Mandates:

- is_same<T,U> is false,
- is_constructible v<T, const U> is true,
- is_assignable<T&, const U> is true,
- is_constructible v<T, std::optional<U>&> is false,
- is_constructible v<T, alloc-optional<U>&> is false,
- is_constructible v<T, std::optional<U>&&> is false,
- is_constructible v<T, alloc-optional<U>&&> is false,
- is_constructible v<T, const std::optional<U>&> is false,
- is_constructible v<T, const alloc-optional<U>&> is false,
- is_constructible v<T, const std::optional<U>&&> is false,
- is_constructible v<T, const alloc-optional<U>&&> is false,
- is_convertible v<std::optional<U>&, T> is false,
- is_convertible v<alloc-optional<U>&, T> is false,
- is_convertible v<std::optional<U>&&, T> is false,
- is_convertible v<alloc-optional<U>&&, T> is false,
- is_convertible v<const std::optional<U>&, T> is false,
- is_convertible v<const alloc-optional<U>&, T> is false,
- is_convertible v<const std::optional<U>&&, T> is false,
- is_convertible v<const alloc-optional<U>&&, T> is false,
- is_assignable v<T&, std::optional<U>&> is false,
- is_assignable v<T&, alloc-optional<U>&> is false,
- is_assignable v<T&, std::optional<U>&&> is false,
- is_assignable v<T&, alloc-optional<U>&&> is false,
- is_assignable v<T&, const std::optional<U>&> is false,
- is_assignable v<T&, const alloc-optional<U>&> is false,
- is_assignable v<T&, const std::optional<U>&&> is false, and
- is_assignable v<T&, const alloc-optional<U>&&> is false

```
template<class U> alloc-optional& operator=(const std::optional<U>& rhs);
```

Effects: The result of the expression `bool(rhs)` remains unchanged. See Table

Table — `alloc-optional operator=(const std::optional<U>&)` effects

	<u>*this contains a value</u>	<u>*this does not contain a value</u>
<u>rhs contains a value</u>	<u>assigns <code>std::move(*rhs)</code> to the contained value</u>	<u>initializes the contained value by uses-allocator construction with allocator <code>polymorphic_allocator<>(_M_pmr)</code> and <code>std::move(*rhs)</code> as the constructor argument.</u>
<u>rhs does not contain a value</u>	<u>destroys the contained value by calling <code>val->T::~~T()</code>.</u>	<u>no effect</u>

Returns: `*this`.

Ensures: `bool(rhs) == bool(*this)`.

Remarks: If any exception is thrown, the result of the expression `bool(*this)` remains unchanged. If an exception is thrown during the call to `T`'s constructor, no effect. If an exception is thrown during the call to `T`'s assignment, the state of `*val` is determined by the exception safety guarantee of `T`'s assignment.

Mandates:

- is constructible `v<T, U>` is true,
- is assignable `v<T&, U>` is true,
- is constructible `v<T, std::optional<U>&>` is false,
- is constructible `v<T, alloc-optional<U>&>` is false,
- is constructible `v<T, std::optional<U>&&>` is false,
- is constructible `v<T, alloc-optional<U>&&>` is false,
- is constructible `v<T, const std::optional<U>&>` is false,
- is constructible `v<T, const alloc-optional<U>&>` is false,
- is constructible `v<T, const std::optional<U>&&>` is false,
- is constructible `v<T, const alloc-optional<U>&&>` is false,
- is convertible `v<std::optional<U>&, T>` is false,
- is convertible `v<alloc-optional<U>&, T>` is false,
- is convertible `v<std::optional<U>&&, T>` is false,
- is convertible `v<alloc-optional<U>&&, T>` is false,
- is convertible `v<const std::optional<U>&, T>` is false,
- is convertible `v<const alloc-optional<U>&, T>` is false,
- is convertible `v<const std::optional<U>&&, T>` is false,
- is convertible `v<const alloc-optional<U>&&, T>` is false,
- is assignable `v<T&, std::optional<U>&>` is false,
- is assignable `v<T&, alloc-optional<U>&>` is false,
- is assignable `v<T&, std::optional<U>&&>` is false,
- is assignable `v<T&, alloc-optional<U>&&>` is false,
- is assignable `v<T&, const std::optional<U>&>` is false,
- is assignable `v<T&, const alloc-optional<U>&>` is false,
- is assignable `v<T&, const std::optional<U>&&>` is false, and
- is assignable `v<T&, const alloc-optional<U>&&>` is false

template<class U> `alloc-optional& operator=(alloc-optional<U>&& rhs);`

Effects: See Table

Table — `alloc-optional operator=(alloc-optional<U>&&)` effects

	<u>*this contains a value</u>	<u>*this does not contain a value</u>
<u>rhs contains a value</u>	<u>assigns *rhs to the contained value</u>	<u>initializes the contained value by uses-allocator construction with allocator polymorphic allocator<>(_M_pmr) and *rhs as the constructor argument.</u>
<u>rhs does not contain a value</u>	<u>destroys the contained value by calling val->T::~~T().</u>	<u>no effect</u>

Returns: *this.

Ensures: bool(rhs) == bool(*this).

Remarks: If any exception is thrown, the result of the expression bool(*this) remains unchanged. If an exception is thrown during the call to T's move constructor, the state of *rhs.val is determined by the exception safety guarantee of T's move constructor. If an exception is thrown during the call to T's move assignment, the state of *val and *rhs.val is determined by the exception safety guarantee of T's move assignment.

Mandates:

- is same<T,U> is false,**
- is constructible v<T, const U> is true,**
- is assignable<T&, const U> is true,**
- is constructible v<T, std::optional<U>& is false,**
- is constructible v<T, alloc-optional<U>& is false,**
- is constructible v<T, std::optional<U>&& is false,**
- is constructible v<T, alloc-optional<U>&& is false,**
- is constructible v<T, const std::optional<U>& is false,**
- is constructible v<T, const alloc-optional<U>& is false,**
- is constructible v<T, const std::optional<U>&& is false,**
- is constructible v<T, const alloc-optional<U>&& is false,**
- is convertible v<const U&, T> is true,**
- is convertible v<std::optional<U>&, T> is false,**
- is convertible v<alloc-optional<U>&, T> is false,**
- is convertible v<std::optional<U>&&, T> is false,**
- is convertible v<alloc-optional<U>&&, T> is false,**
- is convertible v<const std::optional<U>&, T> is false,**
- is convertible v<const alloc-optional<U>&, T> is false,**
- is convertible v<const std::optional<U>&&, T> is false,**
- is convertible v<const alloc-optional<U>&&, T> is false,**
- is assignable v<T&, std::optional<U>& is false,**
- is assignable v<T&, alloc-optional<U>& is false,**
- is assignable v<T&, std::optional<U>&& is false,**
- is assignable v<T&, alloc-optional<U>&& is false,**
- is assignable v<T&, const std::optional<U>& is false,**
- is assignable v<T&, const alloc-optional<U>& is false,**
- is assignable v<T&, const std::optional<U>&& is false, and**
- is assignable v<T&, const alloc-optional<U>&& is false**

template<class U> alloc-optional& operator=(std::optional<U>&& rhs);

Effects: See Table

Table — alloc-optional operator=(std::optional<U>&&) effects

	<u>*this contains a value</u>	<u>*this does not contain a value</u>
<u>rhs contains a value</u>	<u>assigns *rhs to the contained value</u>	<u>initializes the contained value by uses-allocator construction with allocator polymorphic allocator<>(_M_pmr) and *rhs as the constructor argument.</u>
<u>rhs does not contain a value</u>	<u>destroys the contained value by calling val->T::~~T().</u>	<u>no effect</u>

Returns: *this.

Ensures: bool(rhs) == bool(*this).

Remarks: If any exception is thrown, the result of the expression bool(*this) remains unchanged. If an exception is thrown during the call to T's constructor, the state of *rhs.val is determined by the exception safety guarantee of T's constructor. If an exception is thrown during the call to T's assignment, the state of *val and *rhs.val is determined by the exception safety guarantee of T's assignment.

Mandates:

- is constructible v<T, const U&> is true,**
- is assignable<T&, const U&> is true,**
- is constructible v<T, std::optional<U>&> is false,**
- is constructible v<T, alloc-optional<U>&> is false,**
- is constructible v<T, std::optional<U>&&> is false,**
- is constructible v<T, alloc-optional<U>&&> is false,**
- is constructible v<T, const std::optional<U>&> is false,**
- is constructible v<T, const alloc-optional<U>&> is false,**
- is constructible v<T, const std::optional<U>&&> is false,**
- is constructible v<T, const alloc-optional<U>&&> is false,**
- is convertible v<const U&, T> is true,**
- is convertible v<std::optional<U>&, T> is false,**
- is convertible v<alloc-optional<U>&, T> is false,**
- is convertible v<std::optional<U>&&, T> is false,**
- is convertible v<alloc-optional<U>&&, T> is false,**
- is convertible v<const std::optional<U>&, T> is false,**
- is convertible v<const alloc-optional<U>&, T> is false,**
- is convertible v<const std::optional<U>&&, T> is false,**
- is convertible v<const alloc-optional<U>&&, T> is false,**
- is assignable v<T&, std::optional<U>&> is false,**
- is assignable v<T&, alloc-optional<U>&> is false,**
- is assignable v<T&, std::optional<U>&&> is false,**
- is assignable v<T&, alloc-optional<U>&&> is false,**
- is assignable v<T&, const std::optional<U>&> is false,**
- is assignable v<T&, const alloc-optional<U>&> is false,**
- is assignable v<T&, const std::optional<U>&&> is false, and**
- is assignable v<T&, const alloc-optional<U>&&> is false**

template<class... Args>> T& emplace(Args&&...);

Constraints: is constructible v<T, Args&&...> is true.

Effects: Calls *this = nullopt. Then initializes the contained value by uses-allocator construction with allocator polymorphic allocator<>(_M_pmr) and std::forward<Args>(args)... as the constructor argument.

Ensures: *this contains a value.

Returns: A reference to the new contained value.
Throws: Any exception thrown by the selected constructor of T.
Remarks: If an exception is thrown during the call to T's constructor, *this does not contain a value, and the previous *val (if any) has been destroyed.

template<class U, class... Args> T& emplace(initializer_list<U>, Args&&...);

Constraints: is constructible v<T, initializer_list<U>&, Args&&...> is true.
Effects: Calls *this = nullptr. Then initializes the contained value by uses-allocator construction with allocator polymorphic allocator<>(_M_pmr) and il, std::forward<Args>(args)... as the constructor arguments.
Ensures: *this contains a value.
Returns: A reference to the new contained value.
Throws: Any exception thrown by the selected constructor of T.
Remarks: If an exception is thrown during the call to T's constructor, *this does not contain a value, and the previous *val (if any) has been destroyed.

Swap [alloc-optional.swap]

void swap(alloc-optional& rhs);

Constraints: Lvalues of type T shall be swappable and is move constructible v<T> is true.
Expects: a.get_allocator() == b.get_allocator().
Effects: See Table x

Table x — swap(alloc-optional&) effects

	<u>*this contains a value</u>	<u>*this does not contain a value</u>
<u>rhs contains a value</u>	<u>calls swap>(*this, rhs)</u>	<u>initializes the contained value by uses-allocator construction with allocator polymorphic allocator<>(_M_pmr) and std::move(rhs) as the constructor argument, followed by rhs.val->T::~T(); postcondition is that *this contains a value and rhs does not contain a value</u>
<u>rhs does not contain a value</u>	<u>initializes the contained value by uses-allocator construction with allocator polymorphic allocator<>(_M_pmr) and std::move(*this) as the constructor argument, followed by val->T::~T(); postcondition is that *this does not contain a value and rhs contains a value</u>	<u>no effect</u>

Throws: Any exceptions thrown by the operations in the relevant part of Table x.

Observers [alloc-optional.observe]

const T* operator->() const;
T* operator->();

Expects: *this contains a value.

Returns: val.

Throws: Nothing.

const T& operator*() const&;

T& operator*() &;(.).;

Expects: *this contains a value.

Returns: *val.

Throws: Nothing.

T&& operator*() &&;

const T&& operator*() const&&; &;(.).;

Expects: *this contains a value.

Effects: Equivalent to: return std::move(*val);

explicit operator bool() const noexcept;

Returns: true if and only if contains a value.

bool has value() const noexcept;

Returns: true if and only if contains a value.

const T& value() const&;

T& value() &;

Effects: Equivalent to:

return bool(*this) ? *val : throw bad optional access();

T&& value() &&;

const T&& value() const&&;

Effects: Equivalent to:

return bool(*this) ? std::move(*val) : throw bad optional access();

template<class U> T value or(U&&) const&;

Effects: Equivalent to:

return bool(*this) ? **this : static cast<T>(std::forward<U>(v)).;

Remarks: If is copy constructible v<T> && is convertible v<U&&, T> is false, the program is ill-formed.

template<class U> T value or(U&&) &&;

Effects: Equivalent to:

return bool(*this) ? std::move(**this) : static cast<T>(std::forward<U>(v)).;

Remarks: If is move constructible v<T> && is convertible v<U&&, T> is false, the program is ill-formed.

template<class U> T value or(allocator arg t, const polymorphic allocator<>& a, U&&) const&;tt>

Effects: if *this contains a value, returns an object of type T initialized by uses-allocator construction with allocator a and **this as the constructor argument. Otherwise, returns an object of type T initialized by uses-allocator construction with allocator a and std::forward<U>(v) as the constructor argument.

Remarks: If is copy constructible v<T> && is convertible v<U&&, T> is false, the program is ill-formed.

template<class U> T value or(allocator arg t, const polymorphic allocator<>&, U&&) &&;

Effects: if *this contains a value, returns an object of type T initialized by uses-allocator construction with allocator a and std::move(**this) as the constructor argument. Otherwise, returns an object of type T initialized by uses-allocator construction with allocator a and

std::forward<U>(v) as the constructor argument.

Remarks: If is copy constructible v<T> && is convertible v<U&&, T> is false, the program is ill-formed.

allocator type get_allocator() const noexcept;

Returns: pmr::polymorphic_allocator<>(M_pmr).

Throws: nothing.

Modifiers [alloc-optional.mod]

void reset() noexcept;

Effects: If *this contains a value, calls val->T::~~T() to destroy the contained value; otherwise no effect.

Ensures: *this does not contain a value.

Relational operators [pmroptional.relops]

template<class T, class U> bool operator==(const alloc-optional<T>&, const alloc-optional<U>&);

template<class T, class U> bool operator==(const alloc-optional<T>&, const std::optional<U>&);

template<class T, class U> bool operator==(const std::optional<T>&, const alloc-optional<U>&);

Mandates: The expression *x == *y shall be well-formed and its result shall be convertible to bool. [Note: T need not be Cpp17EqualityComparable. — end note]

Returns: If bool(x) != bool(y), false; otherwise if bool(x) == false, true; otherwise *x == *y.

template<class T, class U> bool operator!=(const alloc-optional<T>&, const alloc-optional<U>&);

template<class T, class U> bool operator!=(const alloc-optional<T>&, const std::optional<U>&);

template<class T, class U> bool operator!=(const std::optional<T>&, const alloc-optional<U>&);

Mandates: The expression *x != *y shall be well-formed and its result shall be convertible to bool.

Returns: If bool(x) != bool(y), true; otherwise if bool(x) == false, false; otherwise *x != *y.

template<class T, class U> bool operator<(const alloc-optional<T>&, const alloc-optional<U>&);

template<class T, class U> bool operator<(const alloc-optional<T>&, const std::optional<U>&);

template<class T, class U> bool operator<(const std::optional<T>&, const alloc-optional<U>&);

Mandates: The expression *x < *y shall be well-formed and its result shall be convertible to bool.

Returns: If !y, false; otherwise if !x, true; otherwise *x < *y.

template<class T, class U> bool operator>(const alloc-optional<T>&, const alloc-optional<U>&);

template<class T, class U> bool operator>(const alloc-optional<T>&, const

`std::optional<U>&);`

`template<class T, class U> bool operator>(const std::optional<T>&, const alloc-optional<U>&);`

*Mandates: The expression $*x > *y$ shall be well-formed and its result shall be convertible to `bool`.*

*Returns: If $!x$, `false`; otherwise if $!y$, `true`; otherwise $*x > *y$.*

`template<class T, class U> bool operator<=(const alloc-optional<T>&, const alloc-optional<U>&);`

`template<class T, class U> bool operator<=(const alloc-optional<T>&, const std::optional<U>&);`

`template<class T, class U> bool operator<=(const std::optional<T>&, const alloc-optional<U>&);`

*Mandates: The expression $*x <= *y$ shall be well-formed and its result shall be convertible to `bool`.*

*Returns: If $!x$, `true`; otherwise if $!y$, `false`; otherwise $*x <= *y$.*

`template<class T, class U> bool operator>=(const alloc-optional<T>&, const alloc-optional<U>&);`

`template<class T, class U> bool operator>=(const alloc-optional<T>&, const std::optional<U>&);`

`template<class T, class U> bool operator>=(const std::optional<T>&, const alloc-optional<U>&);`

*Mandates: The expression $*x >= *y$ shall be well-formed and its result shall be convertible to `bool`.*

*Returns: If $!y$, `true`; otherwise if $!x$, `false`; otherwise $*x >= *y$.*

`template<class T, three_way_comparable_with<T> U>`

`compare_three_way_result t<T,U>`

`operator<=>(const alloc-optional<T>&, const alloc-optional<U>&);`

`template<class T, three_way_comparable_with<T> U>`

`compare_three_way_result t<T,U>`

`operator<=>(const alloc-optional<T>&, const std::optional<U>&);`

`template<class T, three_way_comparable_with<T> U>`

`compare_three_way_result t<T,U>`

`operator<=>(const std::optional<T>&, const alloc-optional<U>&);`

*Returns: If $x \&\& y$, $*x <=> *y$; otherwise `bool(x) <=> bool(y)`.*

Comparison with `nullopt` [`pmroptional.nullopts`]

`template<class T> bool operator==(const alloc-optional<T>&, nullopt t) noexcept;`

Returns: $!x$.

`template<class T> strong_ordering_operator<=>(const alloc-optional<T>&, nullopt t) noexcept;`

Returns: `bool(x) <=> false`.

Comparison with `T` [`pmroptional.comp.with.t`]

`template<class T, class U> bool operator==(const alloc-optional<T>& x, const U& v);`

*Mandates: The expression $*x == v$ shall be well-formed and its result shall be convertible to*

bool. [Note: T need not be Cpp17EqualityComparable. — end note]

Effects: Equivalent to: return bool(x) ? *x == v : false;

template<class T, class U> bool operator==(const T& t, const alloc-optional<U>& v);

Mandates: The expression x == *v shall be well-formed and its result shall be convertible to bool.

Effects: Equivalent to: return bool(v) ? x == *v : false;

template<class T, class U> bool operator!=(const alloc-optional<T>& u, const U& v);

Mandates: The expression *x != v shall be well-formed and its result shall be convertible to bool.

Effects: Equivalent to: return bool(x) ? *x != v : true;

template<class T, class U> bool operator!=(const T& t, const alloc-optional<U>& v);

Mandates: The expression x != *v shall be well-formed and its result shall be convertible to bool.

Effects: Equivalent to: return bool(v) ? x != *v : true;

template<class T, class U> bool operator<(const alloc-optional<T>&, const U&);

Mandates: The expression *x < v shall be well-formed and its result shall be convertible to bool.

Effects: Equivalent to: return bool(x) ? *x < v : true;

template<class T, class U> bool operator<(const T& t, const alloc-optional<U>& v);

Mandates: The expression x < *v shall be well-formed and its result shall be convertible to bool.

Effects: Equivalent to: return bool(v) ? x < *v : false;

template<class T, class U> bool operator>(const alloc-optional<T>&, const U&);

Mandates: The expression *x > v shall be well-formed and its result shall be convertible to bool.

Effects: Equivalent to: return bool(x) ? *x > v : false;

template<class T, class U> bool operator>(const T& t, const alloc-optional<U>& v);

Mandates: The expression x > *v shall be well-formed and its result shall be convertible to bool.

Effects: Equivalent to: return bool(v) ? x > *v : true;

template<class T, class U> bool operator<=(const alloc-optional<T>&, const U&);

Mandates: The expression *x <= v shall be well-formed and its result shall be convertible to bool.

Effects: Equivalent to: return bool(x) ? *x <= v : true;

template<class T, class U> bool operator<=(const T& t, const alloc-optional<U>& v);

Mandates: The expression x <= *v shall be well-formed and its result shall be convertible to bool.

Effects: Equivalent to: return bool(v) ? x <= *v : false;

template<class T, class U> bool operator>=(const alloc-optional<T>&, const U&);

Mandates: The expression *x >= v shall be well-formed and its result shall be convertible to bool.

Effects: Equivalent to: return bool(x) ? *x >= v : false;

template<class T, class U> bool operator>=(const T& t, const alloc-optional<U>& v);

Mandates: The expression $x \geq *v$ shall be well-formed and its result shall be convertible to `bool`.

Effects: Equivalent to: `return bool(v) ? x >= *v : true;`

`template<class T, three_way_comparable_with<T> U>`

`compare_three_way_result t<T,U>`

`operator<=>(const alloc_optional<T>& x, const U&v);`

Effects: Equivalent to: `return bool(x) ? *x >= v : strong_ordering::less ;`

Specialized algorithms [pmroptional.specalg]

`template<class T> void swap(alloc_optional<T>& x, alloc_optional<T>& y) noexcept(see below);`

Effects: Calls `x.swap(y)`.

Mandates: `is_move_constructible v<T>` is true and `is_swappable v<T>` is true.

`template<class T> constexpr optional<decay_t<T>> make_optional(T&&);`

Returns: `optional<decay_t<T>>(std::forward<T>(v))`.

Remarks: Specialization of this function template shall be a constexpr function if `uses_allocator<T, polymorphic_allocator<>>::value` is false and the expression `T(std::forward<T>(v))` is a core constant expression.

`template<class T, class... Args> constexpr optional<T> make_optional(Args&&... args);`

Effects: Equivalent to: `return optional<T>(in_place, std::forward<Args>(args)...)`.

Remarks: Specialization of this function template shall be a constexpr function if `uses_allocator<T, polymorphic_allocator<>>::value` is false and the expression `T(std::forward<Args>(args)...) is a core constant expression.`

`template<class T, class U, class... Args> optional<T>`

`constexpr make_optional(initializer_list<U> il, Args&&... args);`

Effects: Equivalent to: `return optional<T>(in_place, il, std::forward<Args>(args)...)`.

Remarks: Specialization of this function template shall be a constexpr function if `uses_allocator<T, polymorphic_allocator<>>::value` is false and the expression `T(il, std::forward<Args>(args)...) is a core constant expression.`

`template<class T, optional<decay_t<T>>`

`alloc_optional(const polymorphic_allocator<>& a, T&&);`

Mandates: `uses_allocator<T, polymorphic_allocator<>>` is true;

Returns: `optional<decay_t<T>>(in_place, allocator arg, a, std::forward<T>(v))`.

`template<class T, class... Args> optional<T>`

`alloc_optional(const polymorphic_allocator<>& a, Args&&... args);`

Mandates: `uses_allocator<T, polymorphic_allocator<>>` is true;

Effects: Equivalent to: `return optional<T>(in_place, allocator arg, a, std::forward<Args>(args)...)`.

`template<class T, class U, class... Args> optional<T>`

`alloc_optional(const polymorphic_allocator<>& a, initializer_list<U> il, Args&&... args);`

Mandates: `uses_allocator<T, polymorphic_allocator<>>` is true;

Effects: Equivalent to: `return optional<T>(in_place, allocator arg, a, il, std::forward<Args>(args)...)`.

Hash support [pmroptional.hash]

template<class T> struct hash<alloc-optional<T>>;

The specialization hash<alloc-optional<T>> is enabled ([unord.hash]) if and only if hash<remove const t<T>> is enabled. When enabled, for an object o of type alloc-optional<T>, if bool(o) == true, then hash<alloc-optional<T>>.(o) shall evaluate to the same value as hash<remove const t<T>>.(*o); otherwise it evaluates to an unspecified value. The member functions are not guaranteed to be noexcept.